DOCUMENT

13

Assignment Report

How each assignment item was met

Stack: Python 3.13 / FastAPI (a port of the workshop's ASP.NET Core demo), pytest, mutmut 3.8, Playwright, Spec Kit, Claude Code (Sonnet 5.5). **Demo video:** Watch the demo video (8-10 min)

1. What was built

Role-based approval limits in the invoice approval service:

Amount	Who may approve	Approvals
up to 50,000.00	Manager or Director	1
up to 500,000.00	Director	1
above 500,000.00	first Manager or Director, second Director	2 distinct users

A refused attempt returns HTTP 403 with a specific reason and **changes nothing** (all checks run before the first mutation; proven by tests that deep-copy the invoice and compare after the refusal). The UI shows the reason and "N of M approvals".

The original repository is C#; with no .NET SDK on the machine and "do it in Python" as the requirement, the backend was ported to FastAPI with the same layering (Domain / Api / Tests), endpoints, `wwwroot` UI and Playwright setup. The 7 original xUnit tests were ported first and kept green before any feature work.

2. How the assignment items were met

#	Requirement	Where
1	Context file	`AGENTS.md` (approval rules table, layering, test/mutation/e2e commands, review expectations); `CLAUDE.md` imports it
2	Skills	`.claude/skills/approval-rules`, `review-findings`, `run-checks` (+ Spec Kit's `speckit-*` skills)
3	Hooks with structured output	`.claude/settings.json` -> `.claude/hooks/`: `log_event.py` (one fixed-schema JSON line per event), `check_summary.py` (every pytest / mutmt / Playwright run becomes `CHECK <name> | metrics | result=pass/fail`), `validate_review.py` (blocks a `pr-reviewer` answer that is not the findings JSON schema)
4	Spec-driven tool	Spec Kit: `.specify/` (installed with `specify init --integration claude`), `specs/001-invoice-approval-limits/` (spec with clarifications, plan, research, data model, contract, quickstart, checklist, tasks)
5	Implement in steps	See section 5 (commit plan): port -> context/agents -> spec -> single-approver limits -> two-approver rule -> e2e -> review fixes
6	Meaningful tests	`backend/tests/` (177 tests) + `e2e/tests/over-limit-rejection.spec.ts`; section 4
7	Review own PR	`docs/pr-review.md`: 11 findings, each fixed / dismissed / deferred with reasons

#	Requirement	Where
8	Log of AI interactions	`logs/ai-interactions.jsonl` (written by the hooks; 27 fixed keys per line) and the readable `logs/ai-interactions-summary.md`
9	Multiple agents / sub-agents	`.claude/agents/` : `spec-writer` , `implementer` , `test-reviewer` , `pr-reviewer` , `mutation-analyst` ; see section 6

3. Mutation scores (mutmut, domain package, run in WSL because mutmut 3 needs `fork()`)

Command: `wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh <label>` . Reports in `docs/mutation/` .

Before (baseline: ported code, the 7 original tests, no approval limits yet):

```
Mutation report: baseline
Tool: mutmut mutmut, version 3.8.0
Total mutants : 36
Killed        : 23
Survived      : 13
Mutation score: 63.9% (killed+timeout / total)
```

First pass after the feature (feature + first 75 tests, before survivor-driven tests):

```
Total mutants : 85
Killed        : 69
Survived      : 16
Mutation score: 81.2%
```

All 16 survivors were MISSING-TEST (the `mutation-analyst` role confirmed none were equivalent): a fake repository whose `update()` was a no-op (so "forgot to save" passed), unasserted `approved_at` , naive timestamps, and unasserted user-facing error messages.

After (final):

```
Mutation report: after
Tool: mutmut mutmut, version 3.8.0
Total mutants : 86
Killed        : 86
Survived      : 0
Mutation score: 100.0% (killed+timeout / total)
```

After the SOLID refactor (rules split into classes, collaborators injected, see section 10):

```
Mutation report: after-solid-refactor
Tool: mutmut mutmut, version 3.8.0
Total mutants : 132
Killed        : 132
Survived      : 0
Mutation score: 100.0% (killed+timeout / total)
```

The refactor first left one survivor (99.2%): a `>` to `>=` in the second-approver rule that an earlier rule had been hiding. A boundary test for each rule on its own closed it.

Caveat on comparing: the mutant count rises from 36 to 86 because the new `approval_policy.py` and `approval.py` are in scope. The scope is the whole domain package in both runs; the 100% is for the code as it stands now. Mutation testing measured the domain unit tests only (API tests are not in the mutmut run).

4. Making the tests mean something

- **The two negative rules** (same user twice; non-Director as second approver) are tested at policy, service and HTTP level, including `Dir.One` vs `dir.one`, whether the duplicate is on the new or the stored side, and a refused second attempt followed by a valid one.
- **"Nothing changes"** is a shared helper (`assert_refused_and_unchanged`): deep copy before, expect `ApprovalNotPermittedError`, assert equality after, assert the reason text, and assert the repository saw no update.
- **Boundaries** both sides: 50,000.00 / 50,000.01 / 500,000.00 / 500,000.01.
- **Reviewing generated tests:** the `test-reviewer` role judged every test KEEP / STRENGTHEN / REMOVE. It caught weak state-error tests, 2 duplicates, a fake that hid missing persistence and 14 missing cases; all handled (`docs/pr-review.md`).
- **A test that can fail:** the concurrency tests were first written against a repository that returned the live object and passed even with the lock removed (so they proved nothing). They were rewritten around a snapshot-and-delayed-save repository and now fail with the lock disabled (2 failed) and pass with it.
- **Playwright:** one test (page objects, role-based locators): sign in as `demo.manager`, Approvals tab, Approve the 75,000.00 invoice, expect the exact reason alert, expect the invoice still pending in both views.

Results: `python -m pytest backend/tests -q` -> 177 passed. `cd e2e && npm test` -> 1 passed.

5. Commit plan (not yet committed)

The work is complete in the working tree but intentionally **uncommitted**. Suggested order, each commit standing alone (stage by path; messages say what and why):

1. `Port backend from ASP.NET Core to Python/FastAPI (behaviour-identical)` - why: no .NET SDK, assignment is Python; keeps the 7 original tests green. Files: `backend/`, `e2e/playwright.config.ts`, `.github/workflows/ci.yml`, `.gitignore`, `README.md`, `scripts/` (baseline state = `docs/mutation/baseline.txt`).

2. `Add AGENTS.md rules, skills, subagents and AI-interaction logging hooks` - `AGENTS.md`, `.claude/`, `logs/`.
3. `Add Spec Kit specification, plan and tasks for approval limits` - `.specify/`, `specs/`.
4. `Enforce Manager and Director approval limits` - single-approver rules (policy, service, 403 mapping, demo users, tests).
5. `Require two distinct approvers, second a Director, above 500,000` - `Invoice.approvals`, duplicate-user and second-approver rules, tests.
6. `Add Playwright test for a refused over-limit approval and approval progress in the UI`.
7. `Fix PR review findings: atomic approvals, amount validation, narrower 400 mapping, strict tests` - plus `docs/pr-review.md`, `docs/mutation/*`, `ASSIGNMENT.md`.

Note: because the whole tree was built before committing, steps 4 and 5 live in the same files; to split them, stage hunks interactively (`git add -p`) or commit 4+5 together. The steps were built and tested in that order (single-approver first, then two-approver), but the intermediate states were not preserved as commits.

6. Agents, skills and hooks: what actually ran

- **Hooks** are live in this repo: `logs/ai-interactions.jsonl` has one line per prompt, tool call, subagent stop and session stop plus `CheckSummary` lines, all with the same 27 keys (tool and model, agent type and mode, the prompt, skills, sub-agents, MCP and tools used, the AI's suggestion, the human decision, verification evidence, status and files touched; see `docs/guides/10-ai-workflow-guide.md`). A readable summary is in `logs/ai-interactions-summary.md`. Secrets in summaries are redacted. A bug found in testing (a regex reading `passed=82 failed=0` as "82 failed") and one in logging (parallel subagents interleaving partial lines) were fixed.
- **Subagents:** the five agent files are in `.claude/agents/`. Claude Code only registers project agents at startup, so in the session that created them they were not selectable by name. The `test-reviewer`, `pr-reviewer` and `mutation-analyst` roles were therefore run as `general-purpose` subagents that were told to read and follow their agent file (read-only). In a fresh session they are addressable by name, and the `validate_review.py` hook (which keys on the agent name `pr-reviewer`) then enforces the findings schema automatically; for this run the reviewer's output was checked against the same schema by inspection.
- **Spec Kit:** `specify init` created the scaffold and templates. The spec, clarifications, plan and tasks were written to follow those templates by the assistant directly rather than by invoking the `/speckit-*` slash commands one by one.
- **Honest limits:** the early planning and port/baseline work happened before the hooks existed, so those first log entries are marked `source: "reconstructed"` rather than captured live.

7. Does the context file matter? (cold agent versus AGENTS.md)

Experiment. The same prompt (the feature text), plan only, no tools, under 300 words, five fixed headings: where the rules go, the unit tests, what each refusal test asserts, how to run the tests, how to treat review findings.

- **Control (no context):** `claude -p` run from an empty directory, so no `AGENTS.md`, `CLAUDE.md`, code or skills were visible.
- **With context:** a subagent given the finished `AGENTS.md` in its prompt.
- **A failed first attempt, kept for honesty:** my first control was a subagent started inside the project. Subagents load `CLAUDE.md` and `AGENTS.md` automatically, and its answer cited the repo's `run-checks` skill and `pr-reviewer` agent, so it was not a control. I discarded it and re-ran the control from an empty directory (that run is not in the hook log, because the hooks only exist inside the project).

Aspect	Without context	With AGENTS.md
Where rules go	A pure policy function in the domain or service layer, controllers only map errors (generic but right)	`approval_policy.py` called by `ApprovalService`; `main.py` and routers only map `ApprovalNotPermittedError` to 403
Boundary values	Whole units: 49,999 / 50,000 / 50,001 and 500,000 / 500,001	One cent either side: 50,000.00 / 50,000.01 / 500,000.00 / 500,000.01, with `Decimal`
"Same user"	Refused when the same user approves twice	Also case and whitespace variants (`strip().casefold()`)
Admin role	Not mentioned (only an unknown role)	Admin can never approve
What a refusal test asserts	A reason code, status unchanged, approvals unchanged, no save	The exact reason text, a deep copy compared by equality, and that the repository recorded no update
Test command	"Find the runner in the build files"	`python -m pytest backend/tests -q`, then the mutation run
Review findings	Check each against code and spec; fix, decline, or ask	Exactly one verdict each: fixed, dismissed or deferred, with reasons; judge tests by mutation score; small commits
Ambiguity	Raised the inclusive-limit and first-approver questions itself	Already resolved by the rules table

What it shows. A capable agent with no context still got the architecture and the "nothing changes" idea right, and it asked sensible questions. The context file added the repo-specific, checkable things that a reviewer would otherwise have to ask for: cent-level boundaries, username normalisation, the Admin rule, the exact assertions, the real commands and the verdict vocabulary.

Limits of the experiment. One run per arm. `AGENTS.md` contains part of the answer by design, so the difference is partly information, not skill. The with-context agent was also inside the project, so I cannot separate `AGENTS.md` from other project context it may have loaded.

8. Decisions and assumptions (also in the spec's Clarifications)

Limits inclusive; first approver above 500,000 may be Manager or Director, the second must be a different Director; "same user" = trimmed, case-insensitive username; `Approver` role replaced by `Manager`; `Admin`

can reject and delete but never approve; refusal = 403 with `{"error": reason}`.

9. How to run everything

```
pip install -e "backend[dev]"
python -m pytest backend/tests -q
python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
cd e2e && npm install && npx playwright install chromium && npm test
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh after
```

Documentation as PDFs (23 separate documents, including architecture diagrams, a reviewer guide and a UI walkthrough) is in `docs/pdf/`; two slide decks (PowerPoint and PDF) are in `docs/presentations/`; rebuild with `pip install -e "backend[docs]" && python scripts/build_docs.py`. A recording script for the demo video is in `docs/guides/15-demo-video-script.md`.

Demo accounts: `demo.manager`, `demo.director`, `demo.director2`, `demo.admin`, password `ChangeMe123!` (local demo only). Try `INV-2001` (75,000.00) and `INV-2002` (600,000.00).

10. Design: SOLID

After the review, the code was refactored to follow the SOLID principles. Behaviour did not change: all 134 earlier tests passed unchanged, then 43 tests were added (177 in total), every case in `scripts/try_all_cases.py` still passes, and mutation stays at 100% (132 of 132).

Principle	What was wrong	What changed	Proof
Single responsibility	`api/main.py` held app wiring, error mapping, validation wording and the session check	Split into `main.py` (wiring only), `error_handling.py` and `auth/middleware.py`; each approval rule is its own class	Each file has one job; rule classes tested on their own
Open/closed	One if-chain decided every rule, so a new rule meant editing it	Four rule classes plus an `ApprovalPolicy` that asks them in order; limits are data (`ApprovalLimits`)	`test_a_new_rule_plugs_into_the_policy_without_changing_the_existing_rules`
Liskov substitution	The fake repository and the real one were never checked against the same contract, and the fake did not assign ids	One contract test suite runs against both; the fake now behaves like the real repository	`test_repository_contract.py` (16 tests)
Interface segregation	`ApprovalService` received a six-method repository but used two	`ApprovalStore` (find, save), `InvoiceReader`, `InvoiceWriter`	`test_the_service_works_with_a_store_that_offers_only_find_and_save`

Principle	What was wrong	What changed	Proof
Dependenc y inversion	The service imported the concrete policy module and the clock; the login router read the demo-user list directly	The service takes a policy and a clock as arguments; login uses a `UserDirectory`; `create_app` accepts replacements	Tests inject a stub policy, a fixed clock, a one-user directory and different limits

Full explanation with before and after: `docs/guides/19-solid-design.md`.